

SE4050 – Deep Learning

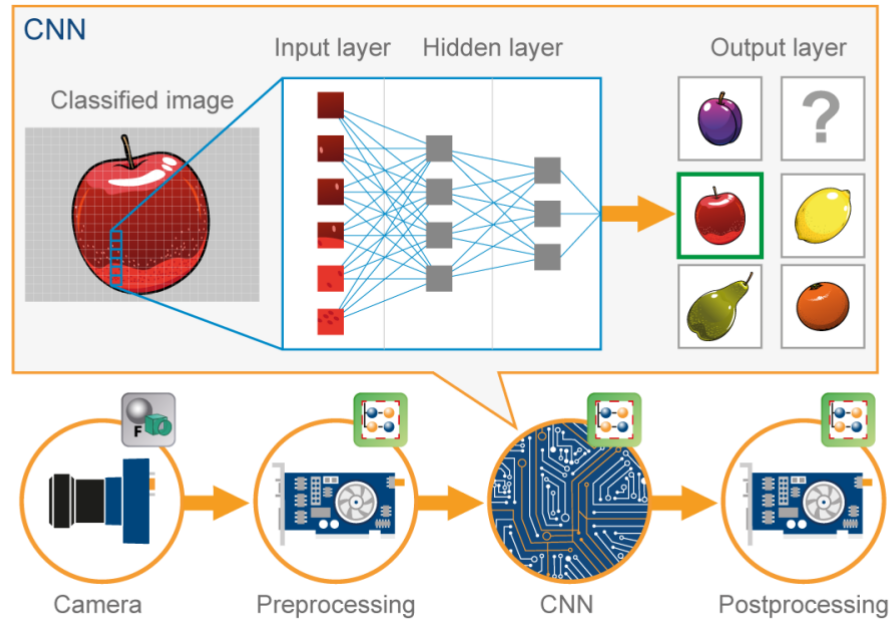
Week 6 – Recurrent Neural Networks and Long-Short term Memory Networks

Dr. Mahima Weerasinghe, *MIET*

Content

- Types of data that we work with
- Spatiotemporal Data
- Recurrent Neural Network
- Long-Short-Term Memory

Types of Data that we work with



Sequential vs non-sequential data

1. Text Data:

- Sentences:** Words in a sentence follow a specific order, which determines meaning (e.g., language translation, sentiment analysis).
- Documents:** Paragraphs or chapters in a book are sequentially structured, impacting the overall narrative.

2. Time Series Data:

- Stock Prices:** Daily closing prices of a stock are dependent on previous days.
- Weather Data:** Temperature or humidity readings over time follow a sequence that can be used for forecasting.

3. Speech and Audio Data:

- Speech Recordings:** The order of phonemes in speech is crucial for recognizing and generating accurate speech.
- Music:** Notes or beats in a piece of music are sequentially organized to form melodies and rhythms.

4. Video Data:

- Video Frames:** The sequence of frames in a video is important for tasks like action recognition or video summarization.

5. Event Logs:

- User Behavior:** Clickstreams or user actions on a website are sequential and can be analyzed to predict future actions.
- Process Logs:** Sequences of events in an industrial process can be analyzed to detect anomalies or predict failures.

Non-sequential data

1. Image Data:

- **Photographs:** Pixels in an image are spatially related rather than sequentially, making CNNs a better fit than RNNs.
- **Medical Imaging:** X-rays, MRIs, and CT scans have spatial structure but no inherent sequence.

2. Tabular Data:

- **Spreadsheets:** Data organized in rows and columns, such as customer databases or sales reports, where each row is independent.
- **Databases:** Records in a relational database where each entry is independent and order doesn't matter.

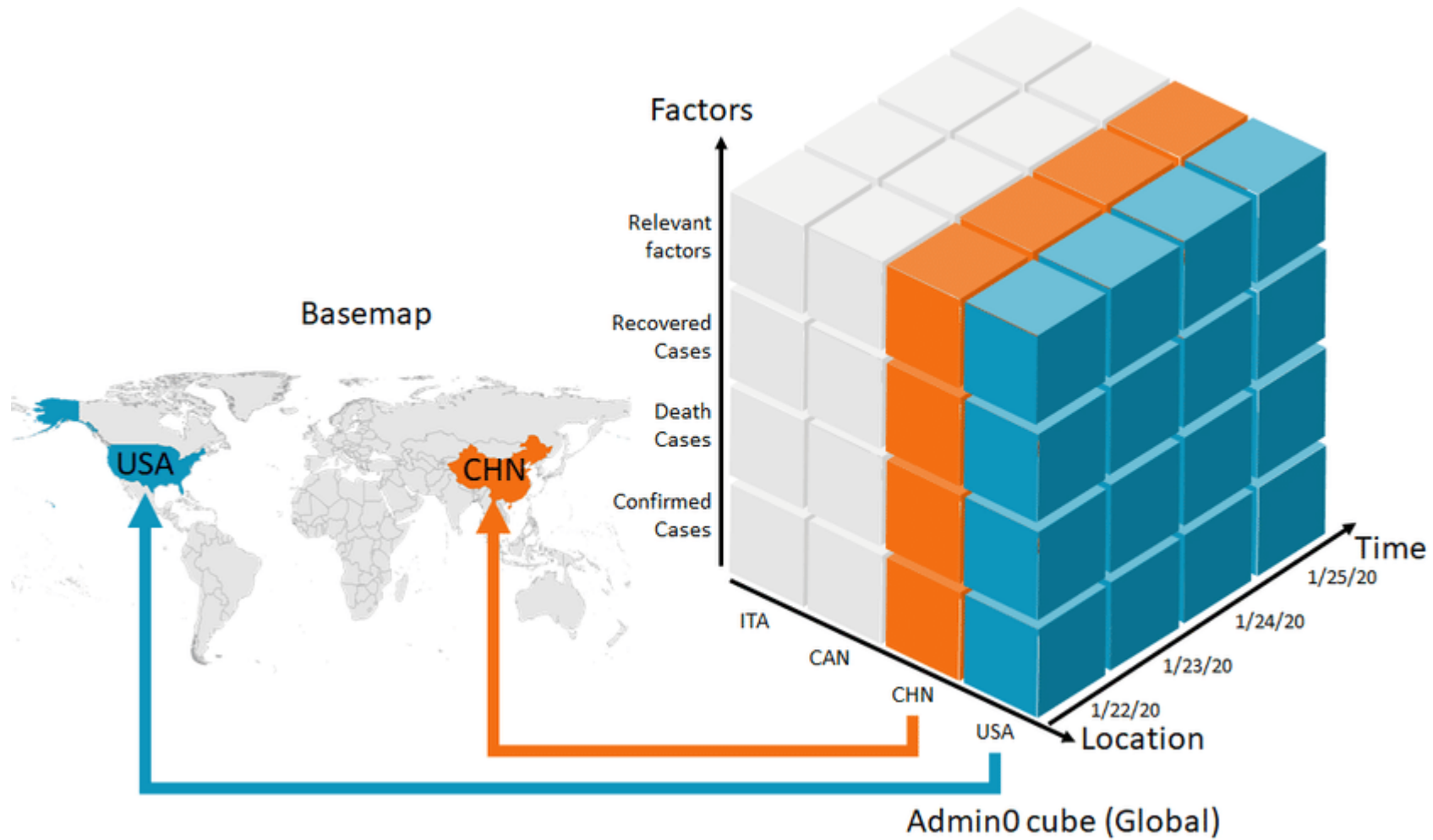
3. Structured Data:

- **Product Listings:** Information about products (e.g., in an online store) that is structured but not sequential.
- **Demographic Data:** Data such as age, income, and education level in surveys, where the order of rows doesn't convey additional information.

4. Graph Data:

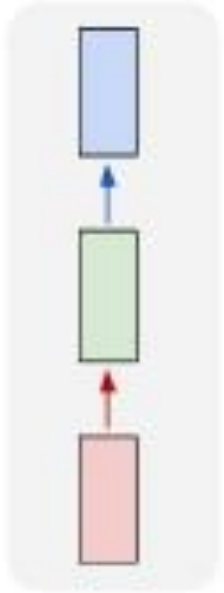
- **Social Networks:** Nodes and edges in a graph structure representing relationships, where the connection pattern matters more than the sequence.
- **Molecular Structures:** Atoms and bonds in chemical compounds represented as graphs rather than sequences.

Spatiotemporal data

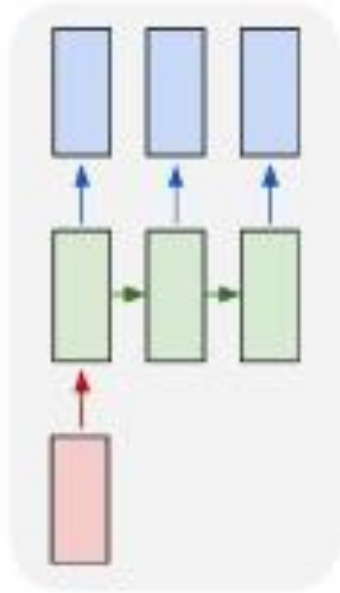


Processing Sequences

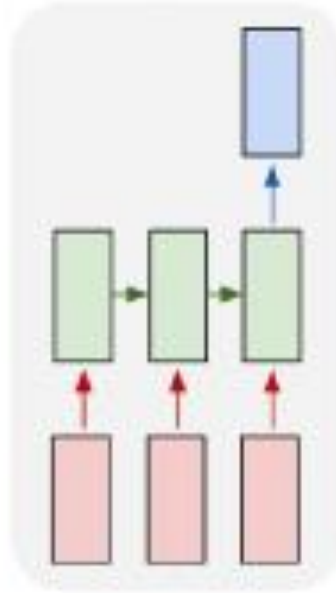
one to one



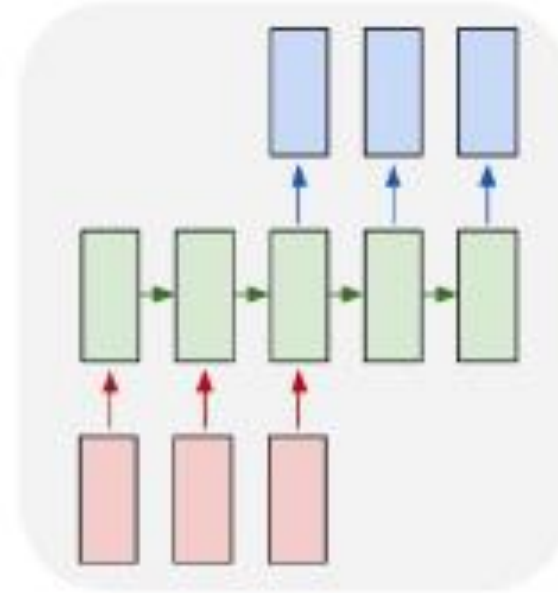
one to many



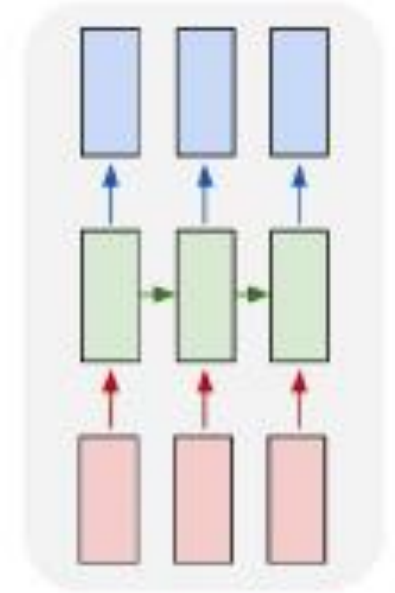
many to one



many to many

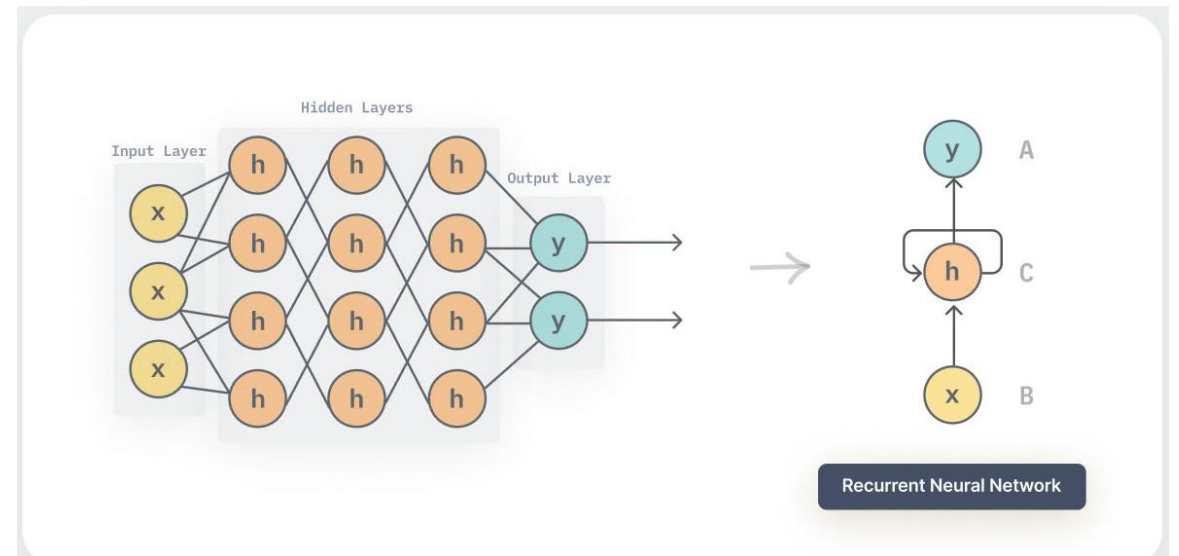


many to many



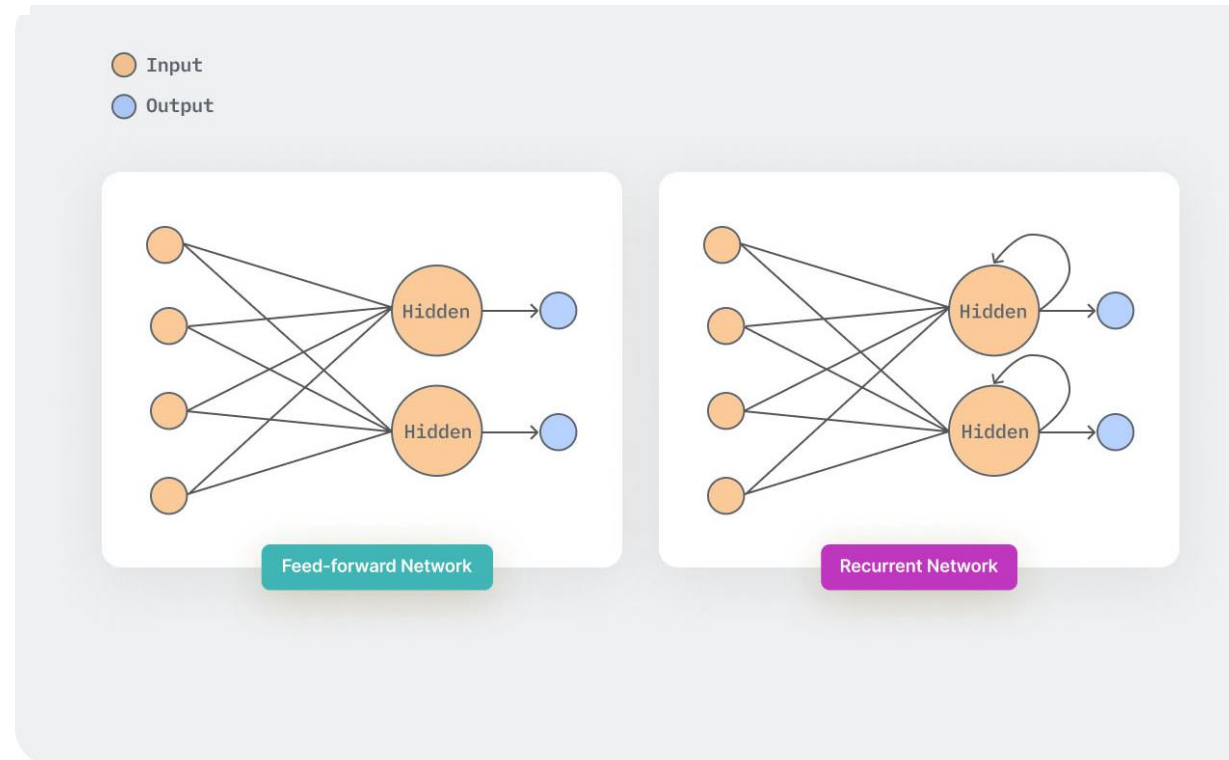
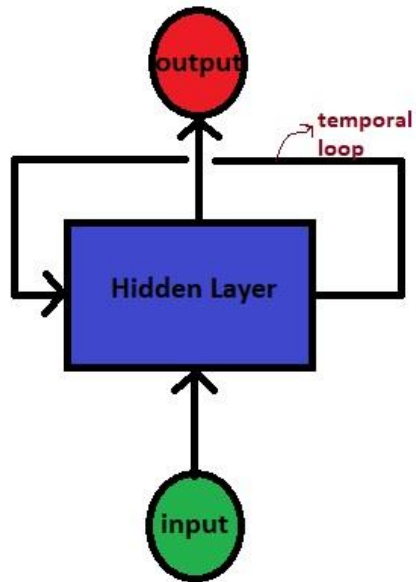
Introduction to Recurrent Neural Networks

When it comes to sequential or time series data, traditional feedforward networks cannot be used for learning and prediction. A mechanism is required to **retain past or historical information** to forecast future values. Recurrent neural networks, or RNNs for short, are a variant of the conventional feedforward artificial neural networks that can deal with sequential data and can be trained to hold knowledge about the past.

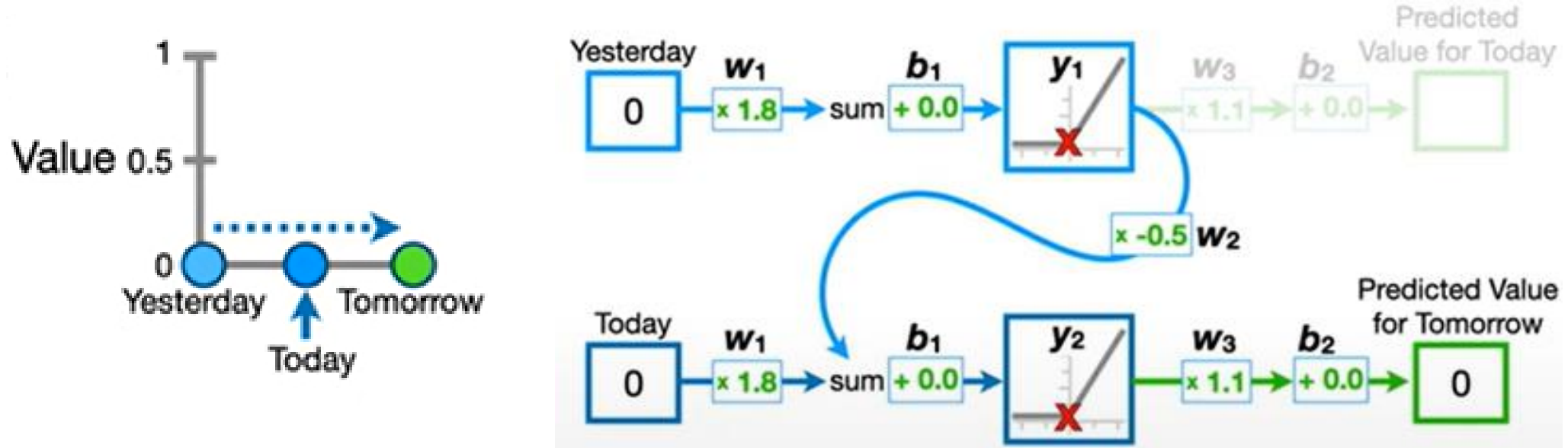


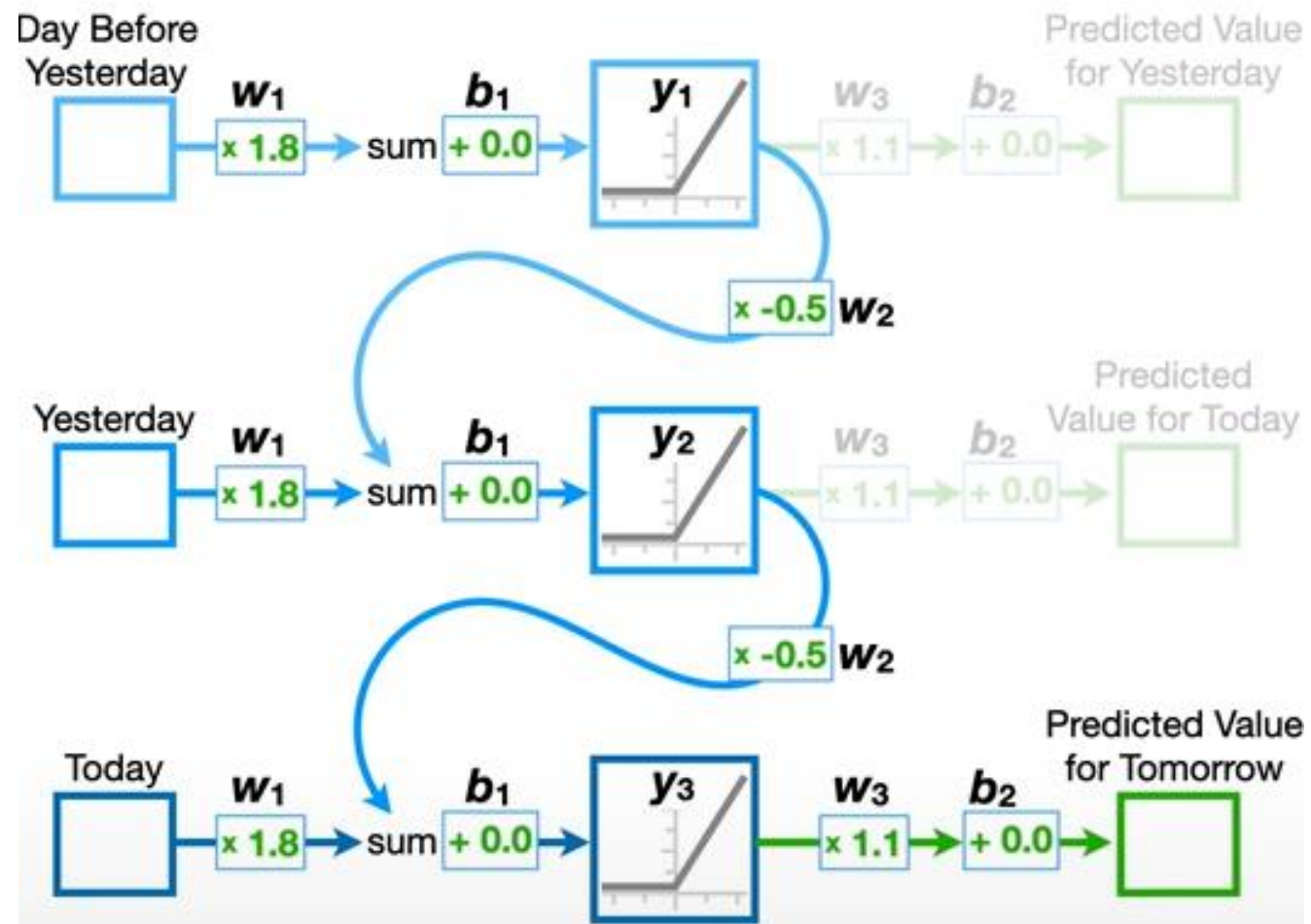
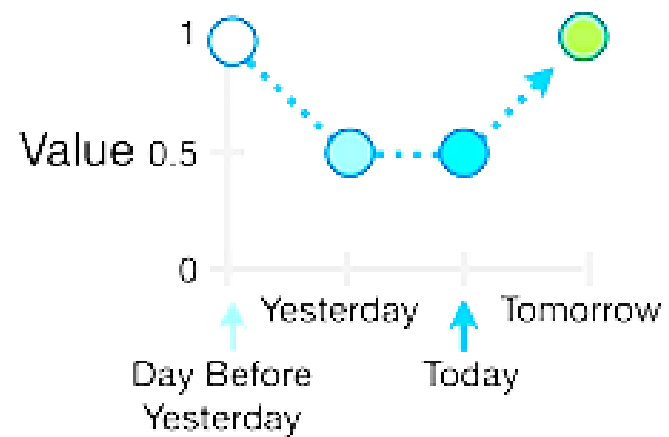
Memory in RNNs

Recurrent Neural Networks have signals traveling in both directions by using feedback loops in the network. Features derived from earlier input are fed back into the network which gives them an ability to memorize. These interactive networks are dynamic due to the ever-changing state until they reach an equilibrium point. These networks are mainly used in sequential autocorrelative data like time series.

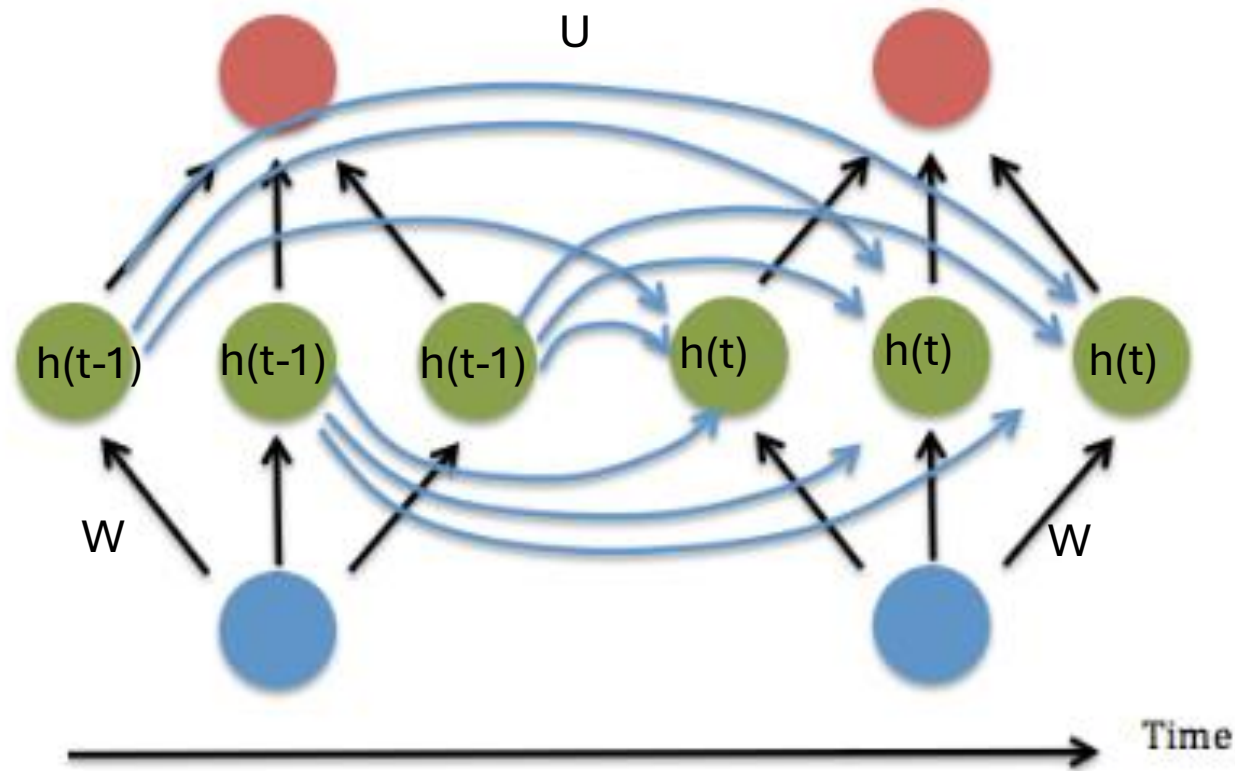


Simple example to understand Unrolling/Unfolding



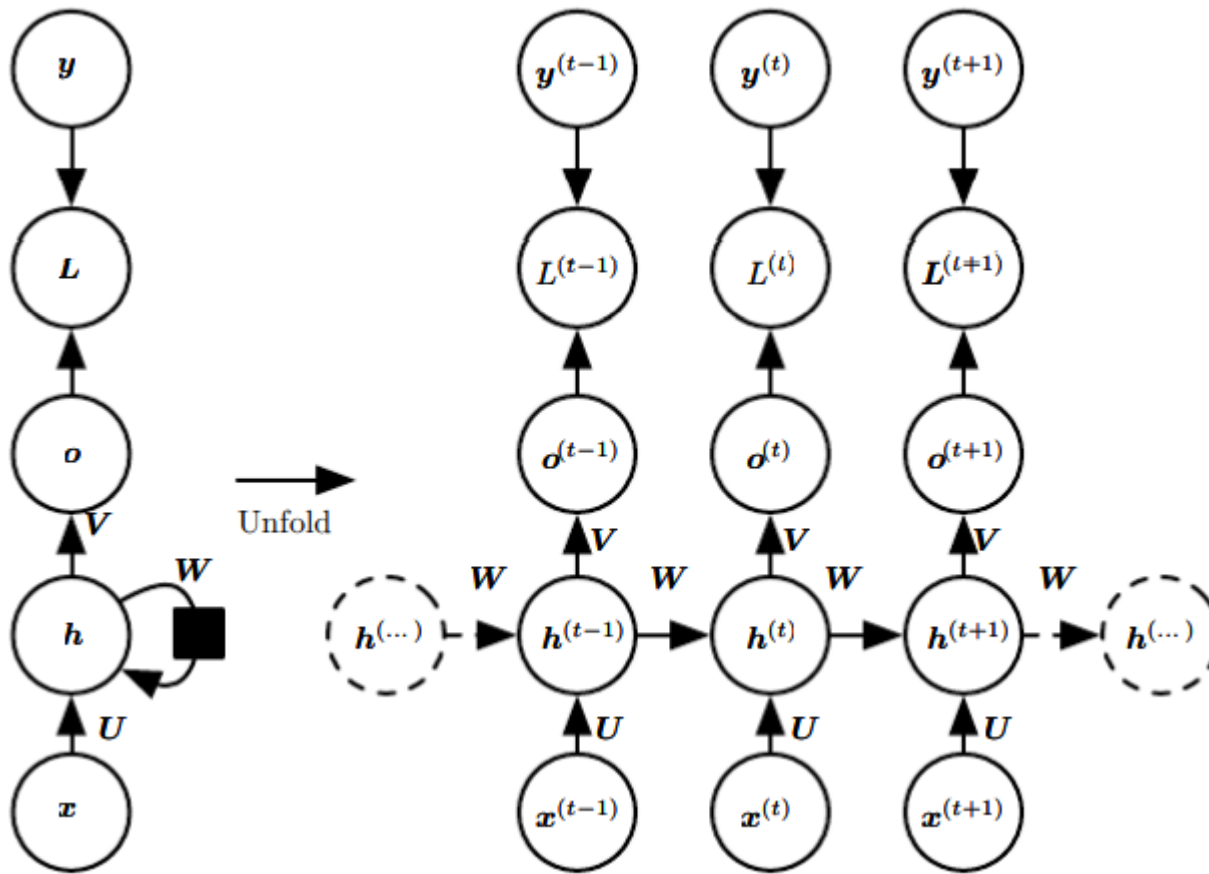


RNN in a known format



$$\vec{h}_t = f(\vec{x}_t, \vec{h}_{t-1}), \text{ where } f(\cdot) \text{ can be written as } \sigma(W\vec{x}_t + U\vec{h}_{t-1}).$$

Architecture – Hidden to hidden



Recurrent Neural Network maps an input sequence $\mathbf{x}$ values to a corresponding sequence of output $\mathbf{o}$ values. A loss $\mathbf{L}$ measure the difference between the actual output $\mathbf{y}$ and the predicted output $\mathbf{o}$. The RNN has also input to hidden connection parametrized by a weight matrix $\mathbf{U}$, hidden to hidden connections parametrized by a weight matrix $\mathbf{W}$, and hidden-to-output connections parametrized by a weight matrix $\mathbf{V}$

Forward Propagation

$$\mathbf{a}^{(t)} = \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)}$$

$$\mathbf{h}^{(t)} = \tanh(\mathbf{a}^{(t)})$$

$$\mathbf{o}^{(t)} = \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)}$$

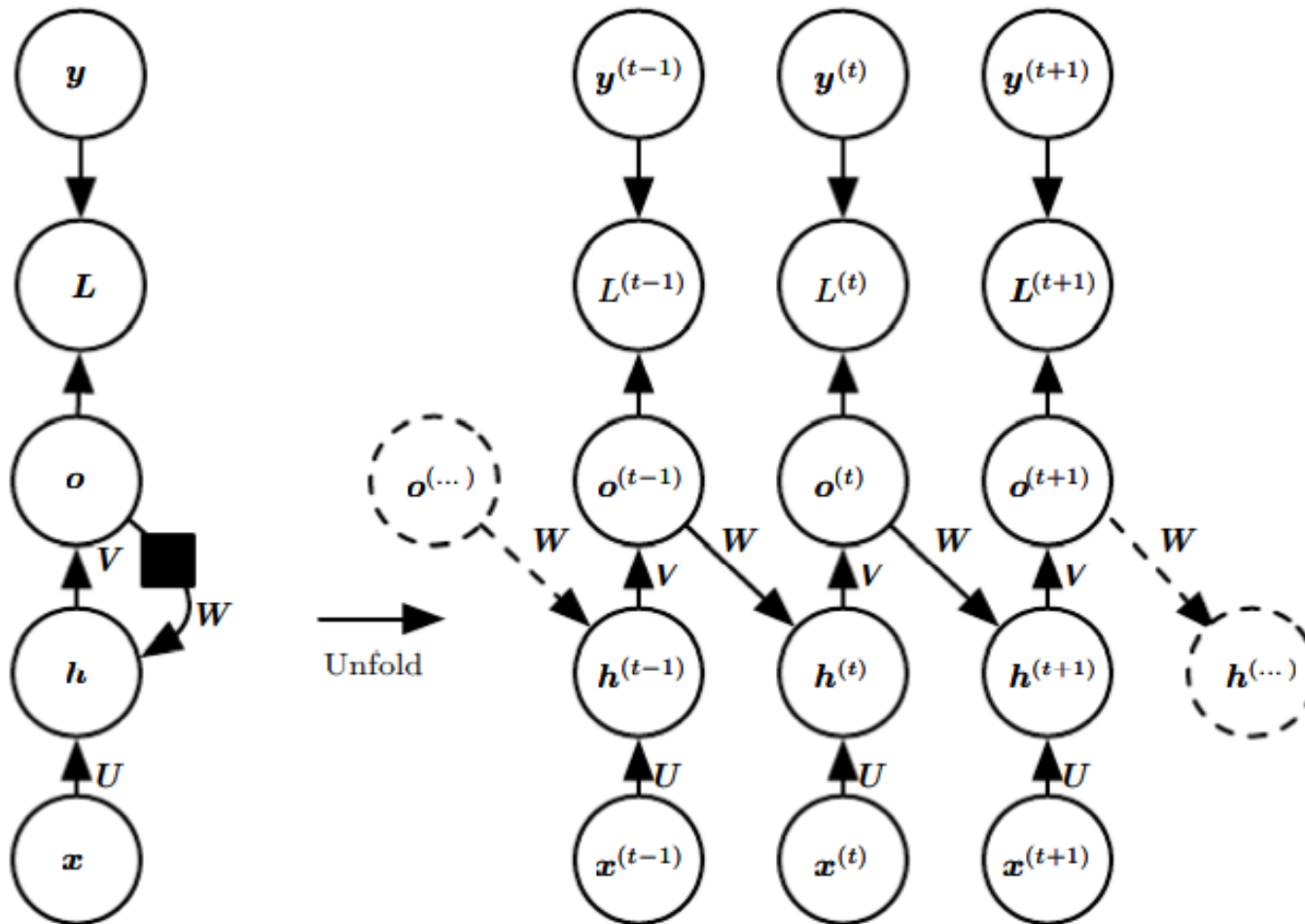
$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)})$$

The above equations are also known as **forwarding propagation** of RNN where the **b** and **c** are the bias vectors and **tanh** and **softmax** are the activation functions. To update the weight matrix **U**, **V**, **W** we calculate the gradient of the loss function for each weight matrix i.e. $\partial L / \partial \mathbf{U}$, $\partial L / \partial \mathbf{V}$, $\partial L / \partial \mathbf{W}$, and update each weight matrix with the help of a back-propagation algorithm.

Weight and Bias Sharing

Concept of Weight Sharing: One of the key features of RNNs is weight sharing across time steps. This means that the same set of weights and biases is used at every time step of the sequence. The weight matrices **U**, **V**, **W** and the bias **b**, are the same for each time step **t**. This shared weight mechanism allows the network to generalize across different parts of the sequence and simplifies the model by reducing the number of parameters.

Architecture – Output to hidden



Note: This types of recurrent neural networks are less powerful and can express a smaller set of functions

Backpropagation in Time (BPTT)

The backpropagation algorithm of an artificial neural network is modified to include the unfolding in time to train the weights of the network. This algorithm is based on computing the gradient vector and is called backpropagation in time or BPTT algorithm for short. The pseudo-code for training is given below. The value of k can be selected by the user for training. In the pseudo-code below, p_t is the target value at time step t :

1. Repeat till the stopping criterion is met:
 1. Set all h to zero.
 2. Repeat for $t = 0$ to $n-k$
 1. Forward propagate the network over the unfolded network for k time steps to compute all h and y
 2. Compute the error as: $e = y_{t+k} - p_{t+k}$
 3. Backpropagate the error across the unfolded network and update the weights

Limitations of RNNs

Simple RNN models usually run into two major issues. These issues are related to gradient, which is the slope of the loss function along with the error function.

Vanishing Gradient problem occurs when the gradient becomes so small that updating parameters becomes insignificant; eventually the algorithm stops learning.

Exploding Gradient problem occurs when the gradient becomes too large, which makes the model unstable. In this case, larger error gradients accumulate, and the model weights become too large. This issue can cause longer training times and poor model performance.

LSTMs – Long Short-Term Memory

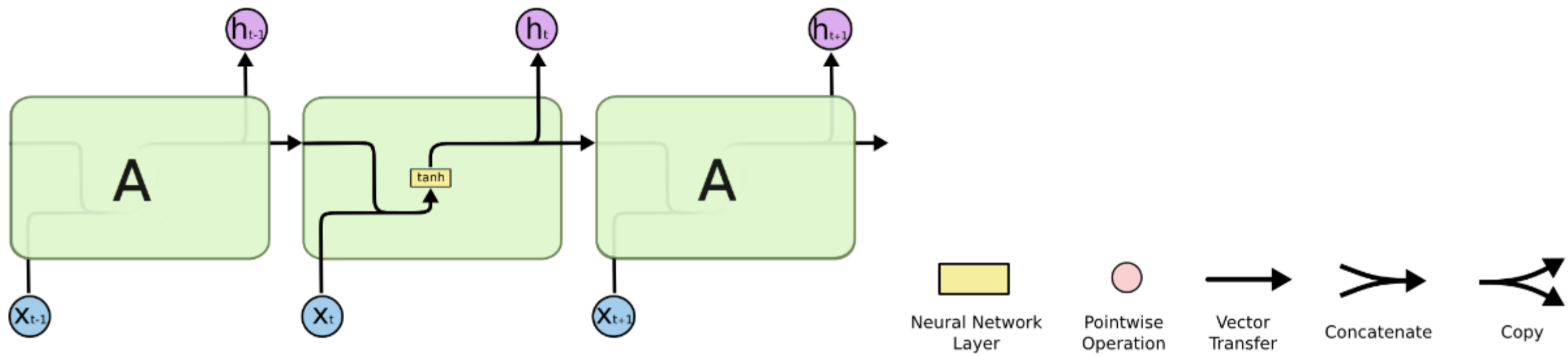
If we are trying to predict the last word in “the clouds are in the *sky*,” we don’t need any further context – it’s pretty obvious the next word is going to be sky. In such cases, where the gap between the relevant information and the place that it’s needed is small, RNNs can learn to use the past information.

But there are also cases where we need more context. Consider trying to predict the last word in the text “I grew up in France... I speak fluent French.” Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of France, from further back. It’s entirely possible for the gap between the relevant information and the point where it is needed to become very large.

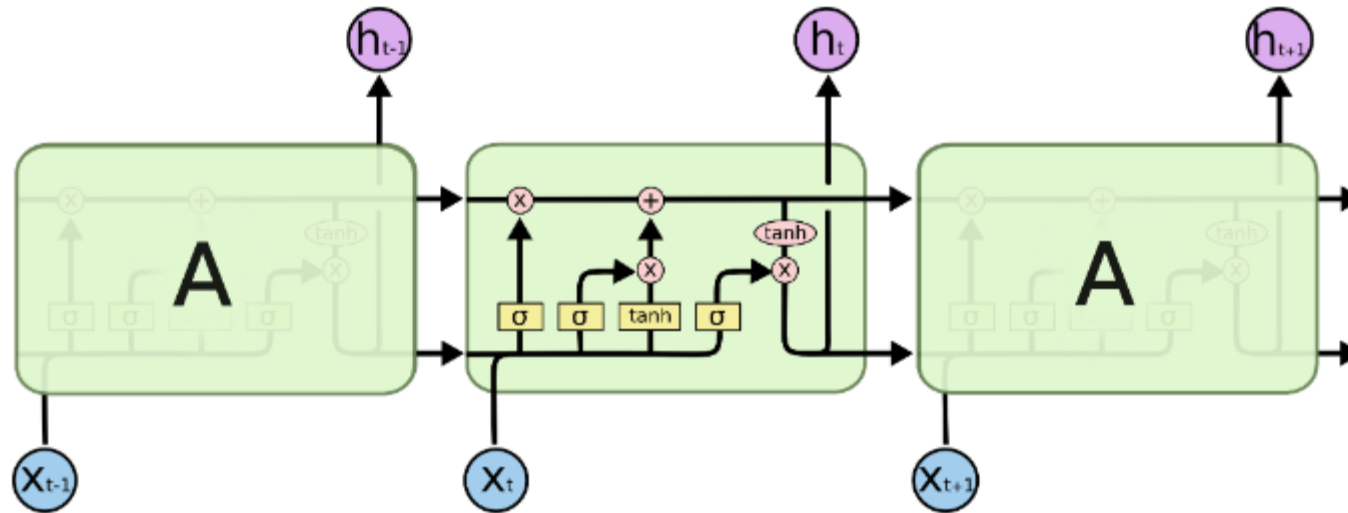
Long Short Term Memory networks – usually just called “LSTMs” – are a special kind of RNN, capable of learning long-term dependencies. They were introduced by [Hochreiter & Schmidhuber \(1997\)](#), and were refined and popularized by many people in following work.¹ They work tremendously well on a large variety of problems, and are now widely used.

LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior, not something they struggle to learn!

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

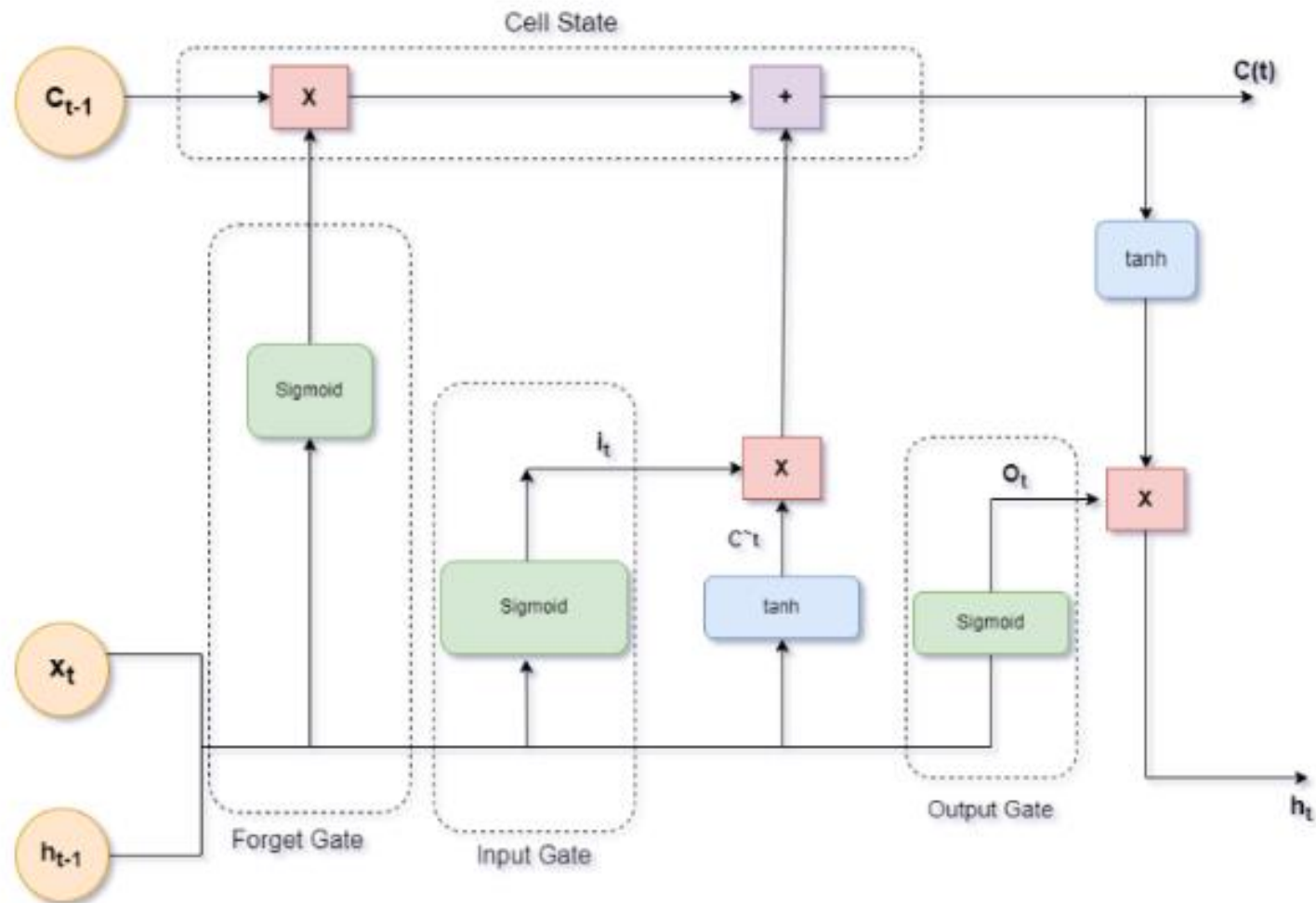


The repeating module in a standard RNN contains a single layer.



The repeating module in an LSTM contains four interacting layers.

LSTM Explanation



LSTM Architecture

1. Architecture of LSTM

The LSTM architecture consists of a cell (the memory part of LSTM), an input gate, an output gate and a forget gate. Each of these components has a specific role in the functioning of the LSTM.

- **Cell:** The cell stores the state of a sequence, so it has the ability to either keep or forget certain information.
- **Input Gate:** It decides the extent of information to be stored in the cell.
- **Output Gate:** It determines what the next hidden state should be.
- **Forget Gate:** It decides what information should be thrown away or kept.

2. Step-by-step Working of LSTM

Let's assume we have a sequence of words ($w_1, w_2, w_3, \dots, w_n$) and we are processing the sequence one word at a time. Let's denote the state of the LSTM at time step t as (h_t, c_t) , where h_t is the hidden state and c_t is the cell state.

- **Step 1:** The LSTM receives the input vector (x_t) and the previous state (h_{t-1}, c_{t-1}).
- **Step 2:** The forget gate (f_t) decides what information to discard from the cell state. It uses the input vector and the previous hidden state to generate a number between 0 and 1 for each number in the cell state c_{t-1} . A 1 represents "completely keep this" while a 0 represents "completely get rid of this".
Forget Gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

$$f_t = \sigma \left(W_f \cdot [h_{(t-1)}, x_t] + b_f \right)$$

- **Step 3:** The input gate (i_t) decides what new information to store in the cell state. It has two parts. A sigmoid layer called the "input gate layer" decides which values we'll update, and a tanh layer creates a vector of new candidate values ($C_t^{\sim}$) that could be added to the state.
Input Gate: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

$$i_t = \sigma \left(W_i \cdot [h_{(t-1)}, x_t] + b_i \right)$$

Input Gate

Candidate Values(Cell State Update): $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$

$$\tilde{C}_t = \tanh \left(W_c \cdot [h_{(t-1)}, x_t] + b_c \right)$$

Cell State Update

•**Step 4:** Update the old cell state (ct-1) to the new cell state (ct). The old cell state is multiplied by ft to forget the things we decided to forget earlier. Then we add the new candidate values, scaled by how much we decided to update each state value.

Cell State(Final Cell State): $ct = f_t * ct-1 + i_t * C_t^{\sim}$

$$C_t = f_t \times C_{(t-1)} + i_t \times \tilde{C}_t$$

Final Cell State

•**Step 5:** Decide the output. This output will be based on our cell state, but will be a filtered version. First, we run a sigmoid layer which decides what parts of the cell state we're going to output. Then, we put the cell state through tanh (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so we only output the parts we decided to.

Output Gate: $ot = \sigma(W_o.[ht-1, xt] + b_o)$

$$o_t = \sigma \left(W_o \cdot [h_{(t-1)}, x_t] + b_o \right)$$

Output Gate

Hidden State: $h_t = o_t * \tanh(c_t)$

$$h_t = o_t \times \tanh(C_t)$$

Hidden State

Advantages of LSTM

- 1.Long-term Dependencies: LSTM is capable of learning long-term dependencies, which is its most significant advantage. It can remember information for long periods, making it perfect for complex tasks such as language translation and speech recognition.
- 2.Handling of Sequential Data: LSTM is designed to handle sequential data. It can process data with time steps of different lengths, unlike traditional neural networks.
- 3.Avoiding Vanishing Gradient Problem: LSTM networks are designed to avoid the long-term dependency problem or the vanishing gradient problem. This means they can learn from data where other models fail.

Disadvantages of LSTM

1.Complexity: LSTM networks are more complex and harder to train than other neural networks.

They require more computational resources and time to train.

2.Overfitting: Due to their complexity and flexibility, LSTM networks are prone to overfitting, especially when dealing with small datasets.

3.Need for Large Datasets: LSTM networks require large amounts of data to train effectively.

They may not perform well on small datasets.

Resources

RNN LSTM Script

<https://colab.research.google.com/drive/1G8GOWb-ZCi2gh0-alD4Kki1HyWB5gvie?usp=sharing>

Vanishing Gradient

<https://medium.com/@amanatulla1606/vanishing-gradient-problem-in-deep-learning-understanding-intuition-and-solutions-da90ef4ecb54>

Introduction to RNNs

<https://machinelearningmastery.com/an-introduction-to-recurrent-neural-networks-and-the-math-that-powers-them/>

Introduction to RNN/LSTM tutorial

<https://www.kaggle.com/code/kcsener/8-recurrent-neural-network-rnn-tutorial>